

Plan d'execution — simplifier Nutrition sans le reecrire

Force Tracker — 11/09/2026. Troisieme et dernier document de la serie : DOUANE-NUTRITION . pdf (les ecritures) -> ARCHI-NUTRITION . pdf (le module) -> **celui-ci, le plan**. Michel : « *on a assez audite. Je veux un chantier progressif, pas une refonte massive.* » **Aucun code n'est ecrit** : ce document propose l'ordre le moins risque et les criteres de passage d'une phase a la suivante.

CE QUE LE PLAN GARDE, ET CE QU'IL CHANGE

Il garde : les **13 portes d'entree** · tous les comportements valides · la provenance · l'historique intact.

Il change : les portes cessent de **reconstruire chacune leur propre forme d'aliment**, les regles communes gagnent un proprietaire unique, et une **douane** apparait avant les 3 ecritures.

Il ne fait PAS : l'historique · les migrations · la reecriture des portes · un gros etat global · aucun changement de comportement aux etapes 1, 2 et 3.

Les deux mesures demandees avant de commencer

Michel : « *ces deux points doivent etre mesures, pas corriges silencieusement* ». Les deux sont **confirmes**. La sonde est rangee dans tools/sonde_fuites_nutrition.js et se rejoue — **elle n'est pas recalculee a la generation de ce PDF** (elle demande un navigateur), donc ce qui suit est la sortie **datee du 11/09/2026**.

Mesure 1 — _bcPaquetG survit bien d'un aliment au suivant

```
ouverture de l'ecran      -> _bcPaquetG = 0      pastille cachee
produit A (boite 410 g)  -> _bcPaquetG = 410   pastille "410 g (le paquet entier)"
produit B (Mes aliments) -> _bcPaquetG = 410   pastille "410 g (le paquet entier)"
                           nom affiche a l'ecran : "Yaourt nature"
```

Confirme. Un yaourt a qui l'app propose « *le paquet entier : 410 g* ». C'est le bug _bcCategories de ft-v1191 **sur une autre variable** : _afOublierAliment nettoie 4 choses et pas celle-la, et la pastille n'est repeinte que par le hub — donc les 3 portes hors hub la laissent a l'ecran.

Mesure 2 — S. savedFoods se perd bien entre deux onglets

```
A ecrit "Pain"           -> disque : ["Pain"]
B (liste perimee) ecrit "Fromage" -> disque : ["Fromage"]      <- Pain perdu
TEMOIN foodLog, meme scenario -> ["Fromage", "Pain"]      <- garde
```

Confirme, et le temoin de controle valide la mesure : la meme manoeuvre sur une liste **fusionnee** garde les deux entrees. Ce n'est donc pas le banc qui deraille.

ET LE CORRECTIF EVIDENT EST FAUX — c'est le point le plus important des deux mesures

Ajouter savedFoods a _fusionListe ferait une **union par nom** — donc **retirer une etoile dans l'onglet A serait annule par la liste perimee de B**.

Les 5 listes deja fusionnees sont des **journaux qui ne font qu'AJOUTER** ; les favoris, eux, **se suppriment couramment**. Une fusion par union sur une liste ou l'on retire est une *resurrection, pas une protection*. Il faut un horodatage ou une pierre tombale — **c'est une decision produit, pas une rustine**, et elle n'est pas prise ici.

Phase 0 — ce qui est mesure aujourd'hui, avant toute extraction

	quoi	pourquoi maintenant
0a	<code>_bcPaquetG</code> remis a zero dans <code>_afOublierAliment</code>	1 ligne + 1 temoin. Doit passer AVANT : la phase 2 change les portes, et la comparaison avant/apres deviendrait impossible a lire
0b	<code>savedFoods</code> multi-onglets	mesure, PAS corrige — ecrit dans <code>docs/JOURNAL-DE-TEST.md</code> avec la raison ci-dessus. Attend une decision

Critere de passage vers la phase 1 : 0a livre · la mutation qui retire la ligne fait rougir **exactement 1** temoin · passe complete au total attendu.

Etape 1 — le constructeur canonique

Deux objets, pas un — c'est la seule subtilite de l'etape, et l'ignorer melangerait la reference pour-100 g avec la ligne consommee.

	fonction	remplace
1a	<code>_ref100(name, kcal, prot, carbs, fat)</code>	les 8 <code>_bcNutr={...}</code> + les 4 traductions
1b	<code>_alimentCanonique(src)</code>	les 5 constructions de <code>{name, kcal, ..., portionWeightG}</code>

Detail — etape 1

rubrique	contenu
fichiers	<code>app.js</code> seul
ancien flux	chaque porte ecrit <code>{name, kcal100: ...}</code> a la main, puis 4 d'entre elles reecrivent <code>per100:{kcal:_bcNutr.kcal100, ...}</code>
nouveau flux	<code>_ref100()</code> rend un objet qui porte les deux faces : <code>kcal100...</code> (tous les lecteurs actuels marchent sans changement) et <code>per100</code> (les 4 traductions disparaissent)
risque	les 2 portes de reprise ne normalisent pas (lignes 2694, 3853, <code>+P.kcal 0</code>). Si <code>_ref100</code> applique <code>_per100d1</code> partout, une valeur peut changer sur la reprise d'une ancienne ligne
reponse au risque	<code>_ref100(..., {normaliser:false})</code> pour ces 2 portes. On extrait, on n'uniformise pas — uniformiser est une decision a part
temoins AVANT	un instantane : pour les 8 portes, serialiser <code>_bcNutr</code> et <code>_afSrc.per100</code> en JSON, dans un fichier de reference
reussite	l'instantane est identique octet pour octet + passe complete au total attendu, aucune valeur attendue modifiee
mutations	(1) <code>_ref100</code> rend <code>null</code> -> toutes les portes rougissent · (2) <code>_per100d1</code> retire -> les 6 portes normalisees rougissent, les 2 brutes non (preuve que l'exception est reelle) · (3) <code>per100</code> non emis -> les 4 ex-traductions rougissent
rollback	1 <code>commit, git revert</code> . Aucune donnee touchee

Etape 2 — la derivation per100

rubrique	contenu
fichiers	app.js — les 3 blocs (deux dans _provFood, un dans l'ecran modifier)
ancien flux	const f=100/masse; p.per100={kcal:_per100d1(vals.kcal*f), ...} x 3
nouveau flux	_per100Derive(totaux, masse) -> null si masse <= 0
risque	quasi nul, mais pas nul : le bloc de l'ecran modifier fait delete e.per100 quand la masse tombe a 0. Ce delete reste chez l'appelant — la fonction rend null, elle ne supprime rien
temoins AVANT	les 3 chemins existent deja au banc (ft-v1188) ; on ajoute un instantane des 3 sorties
reussite	memes chiffres, y compris les decimales
mutations	(1) Math.round au lieu de _per100d1 -> rougit aux 3 · (2) null remplace par {0, 0, 0, 0} -> rougit sur le poids retire
rollback	git revert

Etape 3 — « quantite utilisable » : deux regles, nommees

ON NE FUSIONNE PAS — ET C'EST MESURE

Sur les 6 ecritures du filtre, **2 acceptent** 'portion' et **4 non**. Et les 4 qui refusent alimentent un **champ en grammes** : on ne peut pas y proposer « 2 portions ». **La difference est réelle, pas un oubli**. Les fusionner casserait une des deux.

Detail — etape 3

rubrique	contenu
nouveau flux	_quantiteUtilisable(q, u, {portions: true false}) — un propriétaire, deux modes explicites , jamais un booleen devine
risque	le vrai risque est de se tromper de mode sur un site . D'ou : un site a la fois, un commit par site
temoins AVANT	une table des 6 sites x 4 entrees (120/g · 2/portion · 0/g · 150/ml) figee en instantane
reussite	les 24 cases identiques
mutations	(1) portions:true partout -> rougit sur les 4 · (2) portions:false partout -> rougit sur les 2 · (3) la fonction rend toujours true -> rougit partout
rollback	git revert, 6 commits separes

Etape 4 — le hub commun, une porte a la fois

rubrique	contenu
ordre	(1) onFoodLabelFile (la copie la plus proche du hub) -> (2) quickFillFood -> (3) _afSuggPrendreLocale
ancien flux	chaque porte refait les lignes du hub en omettant etat · sourceId · _bcPaquetG · _afNoteEtat · _bcProposerDerniere(0) · la carte sante
nouveau flux	la porte appelle _offRemplirFormulaire, puis surcharge ce qui lui est propre
risque	c'est la SEULE etape qui change un comportement : ces portes vont gagner ce qu'elles omettaient. etat : 'tel-que-vendu' sur une reprise serait faux — une ligne reprise n'est pas une fiche fabricant
reponse au risque	le hub prend un parametre etat explicite ; et au passage, les 2 clients qui l'ecrasent aujourd'hui (marque, CIQUAL) cessent de travailler pour rien
temoins AVANT	pour chacune des 3 portes, un instantane complet de _afSrc + du DOM du bloc quantite
reussite	le diff est LU ligne par ligne et chaque ecart est nomme — soit voulu, soit refuse. <i>Un diff qu'on ne lit pas est un diff qu'on accepte a l'aveugle.</i>
mutations	(1) porte remise sur sa copie locale -> rougit · (2) etat pose en dur -> rougit sur la reprise
rollback	1 commit par porte — on annule une porte sans toucher aux deux autres

Etape 5 — la douane

rubrique	contenu
ou	entre les 3 push et S · foodLog — une piece neuve , pas un déplacement de code
entree	un objet deja canonique (etape 1)
elle controle	quantite / unite · per100 · les 4 macros · regles physiques · coherence kcal/macros · provenance · doute · champs obligatoires · perte silencieuse de champs
le point decisif	la liste des champs devient FERMEE : un champ inconnu leve , il ne disparaît plus. <i>C'est ca qui tue la famille des 4 oublis de _provFood.</i>
risque	une douane qui REFUSE peut faire perdre un repas — or la regle d'or n°3 dit <i>zero perte</i>
reponse au risque	la douane MESURE avant de MORDRE : livree d'abord en mode observateur (elle écrit son verdict sur la ligne, elle ne bloque rien). On lit ce qu'elle aurait refuse sur les vraies donnees ; ensuite seulement on decide ce qui devient bloquant
reussite (observateur)	100 % des lignes passent, et les verdicts sont lisibles
mutations	(1) douane court-circuitée · (2) champ inconnu accepte en silence · (3) chaque controle neutralise un par un
rollback	en mode observateur, retirer l'appel suffit — aucun comportement n'en depend

Les phases et leurs criteres de passage

phase	contenu	critere pour passer a la suivante
0	_bcPaquetG corrige · savedFoods mesure et ecrit	mutation qui mord · passe au total attendu
1	etapes 1, 2, 3 — extractions pures	les 3 instantanes identiques octet pour octet · aucune valeur attendue modifiee · toutes les mutations mordent
2	etape 4 — le hub, 3 portes	chaque ecart du diff nomme et justifie · validation iPhone apres CHAQUE porte , pas a la fin
3	etape 5 — la douane en observateur	ses verdicts lus sur les vraies donnees · zero ligne refusee
4	duplications devenues mortes	rien ne se supprime sans une mutation qui prouve que plus personne ne l'appelle (R30 : un retrait s'ecrit avec sa raison)
5	validation iPhone complete	les 13 portes parcourues a la main

LA REGLE QUI GOUVERNE TOUT LE PLAN

On ne passe pas a la phase suivante tant qu'une seule mutation de la phase en cours ne mord pas.

Une mutation qui ne mord pas est **soit du code inutile, soit un trou de temoin** — et c'est exactement cette question qui a trouve de vrais defaults en **ft-v1186, ft-v1187, ft-v1189 et ft-v1191**. Quatre fois de suite, le controle negatif a corrige le temoin avant que le temoin ne certifie du vide.

Recommandation

Phase 0 tout de suite — le paquet de 410 g est un vrai bug, visible a l'ecran, et il tient en une ligne. Puis **l'etape 1a seule** : c'est celle qui rapporte le plus (les 4 traductions et les 8 constructeurs) pour le risque le plus faible, et elle se juge sur un critere binaire — **l'instantane bouge, ou il ne bouge pas**.

CE QUE CE DOCUMENT N'EST PAS

Aucun code n'a ete ecrit, aucune ligne de production n'a change — sauf l'ajout de la sonde de mesure dans `tools/`, qui ne tourne jamais dans l'application.

Honnete sur la methode : les chiffres du code sont recalculés a chaque generation et le generateur **refuse de produire** si la forme du depot change ou si l'une des deux fuites est refermee. **Les deux mesures de fuite, elles, ne sont pas recalculees ici** (elles demandent un navigateur) : le document cite la sortie datee et nomme le fichier pour qu'on puisse la rejouer. *Un chiffre mesure qu'on ne peut pas rejouer est un chiffre qu'il faut croire.*